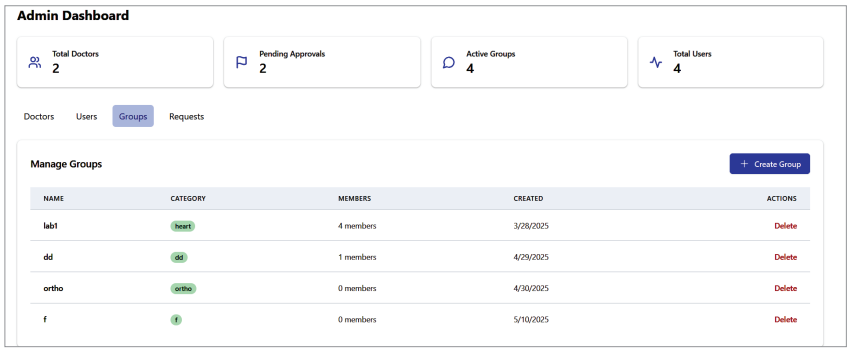

Figure 1: Admin dashboard overview

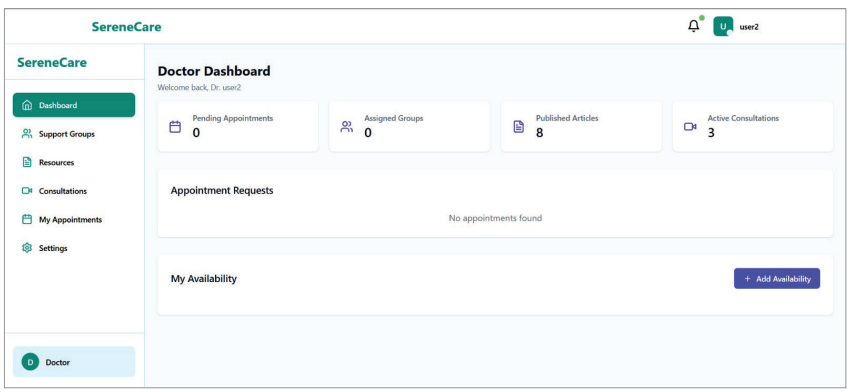

Figure 2: Doctor dashboard interface

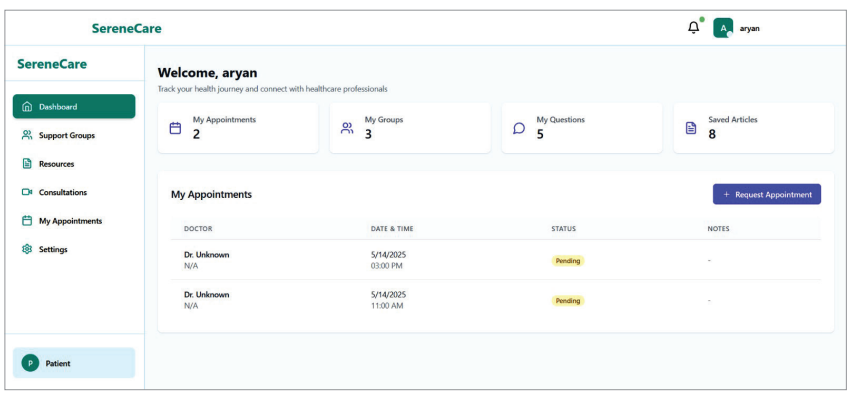

Figure 3: Patient dashboard view

Context helped in dynamically rendering role-based views. Instead of hard-coding separate routes for each user type, we used a central logic to determine what each user sees based on their JWT token data. In short, this UI structure didn't just improve usability — it helped us enforce role-based restrictions and data privacy from the frontend itself.

System architecture: When we started building SereneCare, we knew we needed more than just working code — we needed a structure that could grow with time, handle real-world data securely, and stay organised across multiple features. To achieve that, we leaned on a modular, service-oriented architecture, separating major components into logical units right from the beginning.

This design allowed us to isolate responsibilities across different services: user management, appointments, groups, and shared resources. It also meant that we could work on and test each part of the system without accidentally breaking something else — a huge time-saver during development.

Backend foundation: We built the backend using Node.js with Express.js, mainly for its speed, non-blocking nature, and the large number of libraries available. Instead of having one large API doing everything, we broke the functionality into smaller modules with dedicated routers and middleware.

Each module exposes a clear set of REST APIs that can be consumed by the frontend and are protected by middleware for authentication and role checks. We also included a lightweight API gateway logic, which allows all incoming requests to be routed through a common entry point before being dispatched to the respective module.

Database design with MongoDB: We chose MongoDB as our database mainly for its schema-less flexibility. Working in a healthcare-related domain, we found that patient data, appointment details, and uploaded resources can all have slightly different structures. MongoDB gave us the freedom to evolve our models quickly as new features were added.

We used separate collections for:

- *Users* – storing personal details and roles
- *Appointments* – tracking patient-doctor interactions
- *Groups* – for community discussions and moderation
- *Resources* – documents uploaded by doctors

Each record also includes metadata like timestamps, access flags, and audit logs where necessary.

Frontend architecture: On the frontend, our goal was to keep things modular and responsive. We built the UI using React + TypeScript, which gave us type safety and consistent component behaviour.